

Master Plan — ESP32 AI Club Project

Project: replicate the esp32-ai experiment, verify the published results, improvise on the design, and build a practical use case.

Base repo: <https://github.com/slvDev/esp32-ai>

Goal

1. Reproduce the training runs and verify the published numbers.
2. Verify the on-chip claims on an ESP32-S3 N16R8 board.
3. Improve the design with one improvise experiment.
4. Build a practical use case on the board.
5. Bring every member to a working understanding of neural networks and transformers.

Team slots

Slot	Track	Name (fill in)
R1	ML research	
R2	ML research	
R3	ML research	
R4	ML research	
R5	ML research	
H1	ML + embedded	
H2	ML + embedded	
H3	ML + embedded	
H4	ML + embedded	
H5	ML + embedded	
E1	Embedded only	
E2	Embedded only	

Timeline (8 weeks, 2 months)

Week	Phase	Milestone
1	Host setup	Env, data, tracking sheet, runbook
2	Host training	6 training runs launched in parallel
3	Host eval	Perplexity, PTQ, export, golden
4	Host gate	Verification report, improvise menu
5	Hardware	Board bring-up, on-chip verification
6	Hardware + improvise	Hardware gate, fine-tune experiment
7	Use case	Selection + prototype on hardware
8	Showcase	Demo, report, wrap-up

The published claims to reproduce

Source: RESULTS.md in the repo. Host claims:

Claim	Published value
Perplexity, baseline	12.58
Perplexity, ple	11.41
Perplexity, fatembed	11.94
PLE vs baseline edge	+0.098 nats, +/-0.006, 2 seeds
4-bit PTQ survival	Edge retained 124-128%
Vocab-4096 control	+0.025 nats
Golden match, C vs PyTorch	Max abs diff 0.00001

Hardware claims (verified in weeks 5-6):

Claim	Published value
End-to-end speed	9.88 tok/s
Compute	94.9 ms/token
Stage profile	head 57.6, attn 25.6, ple 8.5, ffn 6.9, input 4.4 ms/token
Flash random read, 512 B row	20.3 us
PSRAM sequential read	60.7 MB/s
SRAM sequential read	240 MB/s
Table cost per token	~0.12 ms (~0.7%)
SRAM free after alloc	294 KB
PSRAM free after alloc	4.22 MB

Verification gates

Host gate, end of week 4:

- Each arm ppl within +/-0.05 of the published value.
- The PLE-vs-baseline edge is positive.
- The 4-bit PTQ edge is retained.
- The golden diff stays at 1e-5 or below.
- Generated text is coherent.

Hardware gate, end of week 6:

- On-chip tok/s within +/-15% of 9.88.
- Bandwidth numbers within +/-15% of published.
- Text generated on the device is coherent.

Note: the repo author states that reproduction is approximate. Runs land on comparable numbers, not identical bytes. The gates above allow for that.

Shared rules

- Fork the repo into the club GitHub account. Work on branches.
- One tracking sheet (Google Sheets) holds every claim, our value, and the owner. R2 owns it.
- Weekly meetup: one member presents that week's learning topic. Each member presents once.
- Each member writes one weekly deliverable. Deliverables live in a shared Drive folder.

- Board time is scheduled. E1 owns the schedule.

Colab runbook (summary)

- Each training run uses one member's Google account. 6 runs = 6 accounts in parallel.
- A run takes roughly 0.5-2 h on a T4 GPU.
- Save checkpoints to Google Drive after every 1000 steps.
- If the session dies, resume from the last checkpoint. Never restart from zero.
- Kaggle is the fallback. It gives 30 GPU-hours per week.
- Dataset: first 300 MB of TinyStories. Download once, cache in Drive.

Training commands (run from the repo root):

```
uv sync
uv run python -m research.tinystories.prepare --vocab 32768
uv run python -m research.tinystories.train --arm baseline --vocab 32768 --steps 3000 --target-core 559000 --seed 0
```

Replace `--arm` and `--seed` per run. The 6 runs:

Run	Arm	Seed	Owner
1	baseline	0	R1
2	ple	0	R2
3	fatembed	0	R3
4	baseline	1	R4
5	ple	1	R5
6	fatembed	1	H1

Analysis and evaluation:

```
uv run python -m research.tinystories.analyze --tag cleandeploy --expect-arms baseline,ple,fatembed --expect-seeds 2
uv run python -m research.tinystories.quantize_eval --tag cleandeploy --seed 0
uv run python -m research.tinystories.quantize_eval --tag cleandeploy --seed 0 --group 128 --fp16-scales
uv run python -m research.tinystories.sample --run runs/ple-cleandeploy-s0.pt --tokenizer data/tinystories/vocab-32768/tokenizer.json
```

Hardware phase (weeks 5-6)

- Board: ESP32-S3 N16R8 (16 MB flash, 8 MB PSRAM, 512 KB SRAM).
- Toolchain: Arduino ESP32 core 3.3.10 or ESP-IDF.
- Deploy commands:

```
bash scripts/fetch_model.sh tinystories
bash scripts/deploy.sh tinystories
```

- The same two commands work for `barista`.
- With 1 board: E1 owns the board. Others get scheduled windows.
- With 2 boards: E1 owns board A. E2 owns board B.
- Weeks 1-4 have no board work. Board work starts in week 5.

Improvise directions (choose in week 5)

1. Fine-tune the model on a new domain. Build a small dataset, retrain, flash it.
2. Interactive serial prompting on the board.
3. SIMD or int4-in-PSRAM head experiment.
4. PLE table scaling probe (wider rows vs more rows).
5. New tokenizer or vocabulary size.

The repo's own next steps list directions 1-3. The team picks one in week 5.

Use case (weeks 7-8)

Criteria for the use case:

- It works with a TinyStories-class model (short text, no world knowledge).
- It runs on one board, offline.
- It is demo-able at the club showcase.
- It serves a real need (educational, accessibility, low-bandwidth, hobby).

Candidates to discuss: offline story companion, flashcard generator, recipe assistant, plant-care assistant, exam-question practice device, low-bandwidth chat toy.

PDF conversion

Install pandoc and a PDF engine, then:

```
for f in *.md; do pandoc "$f" -o "${f%.md}.pdf"; done
```

Each member receives their own .md and .pdf file.

Risks

Risk	Mitigation
Colab disconnects	Checkpoint + resume discipline. H5 runbook.
Colab GPU quota	6 accounts in parallel. Kaggle fallback.
Dataset drift	The repo does not pin the dataset revision. Record the fetch date.
Approximate reproduction	Gates allow +/-0.05 ppl and +/-15% speed.
1 board only	Board schedule. Host work continues in parallel.
Beginners (no NN knowledge)	Curriculum starts at neural-net fundamentals in week 1.

Key resources

- Repo: <https://github.com/slvDev/esp32-ai>
- RESULTS.md: <https://github.com/slvDev/esp32-ai/blob/main/RESULTS.md>
- TinyStories paper: <https://arxiv.org/abs/2305.07759>
- Gemma 3n (PLE): <https://ai.google.dev/gemma/docs/gemma-3n>
- Attention Is All You Need: <https://arxiv.org/abs/1706.03762>
- 3Blue1Brown neural networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- TinyStories dataset: <https://huggingface.co/datasets/roneneldan/TinyStories>